Release Management & Continuous Deployment

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

01 May 2026

Today's Agenda

Agenda

The Journey So Far

We are nearing the end of our Software Engineering journey:

- **Weeks 1-6:** Planning, Architecture, and Design.
- **Weeks 7-9:** Containerisation and Automated CI Pipelines.
- **Week 12:** Refactoring and Technical Debt.

Your code is clean, containerised, and tested. The CI pipeline is green. Now comes the most terrifying part of software engineering: **Putting it in front of actual users.**

Deployment's Problems

Why is deployment historically so stressful?

- **Downtime:** To swap in new code, you often have to shut the server down first — meaning real users see an error page during the update (a "Maintenance Window").
- **The Unknown:** You test in a "Staging" environment that *should* mirror live, but small differences (OS version, config, data volume) mean bugs only appear in Production — in front of real users.
- **The Rollback Nightmare:** If the new version corrupts the database (e.g. renames a column), you cannot simply restart the old version — it now reads data in a format it does not understand.

The Goal of Release Management

To make software deployments boring, routine, and virtually invisible to the end-user.

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Apply** Semantic Versioning (SemVer) to track software releases.
2. **Differentiate** between Continuous Delivery and Continuous Deployment (CD).
3. **Evaluate** deployment mechanics (Blue/Green, Canary Releases).
4. **Control** feature exposure using Feature Flags (release control).

Agenda

What is a Release?

A **Release** is a specific, immutable snapshot of your software that is packaged and distributed to users or servers.

If you just push code to the ‘main’ branch continuously, how do you know *exactly* which version of the code caused the production server to crash at 3:00 AM?

We solve this through **Versioning** and **Git Tags**.

Semantic Versioning (SemVer)

The industry standard for versioning software is **SemVer**. It uses three numbers separated by dots: **MAJOR.MINOR.PATCH** (e.g., `v2.4.1`).

- **MAJOR (X.y.z):** You make incompatible API changes. (If you update this, clients using the old API *will* break).
- **MINOR (x.Y.z):** You add functionality in a backwards-compatible manner. (Old clients can still use the system).
- **PATCH (x.y.Z):** You make backwards-compatible bug fixes.

Note: $v0.x.x$ is reserved for initial development (anything can break at any time).

Think-Pair-Share: Semantic Versioning

Scenario: Your current API version is `v1.4.2`.

Discuss with your neighbor what the *new* version number should be in each of the following three independent scenarios:

1. You refactor a database query to make it 10x faster. The API inputs/outputs do not change.
2. You completely delete the `/api/legacy_stats` endpoint because nobody uses it anymore.
3. You add an optional `?format=json` parameter to the `/api/move` endpoint.

Think-Pair-Share: Semantic Versioning

Scenario: Your current API version is `v1.4.2`.

Discuss with your neighbor what the *new* version number should be in each of the following three independent scenarios:

1. You refactor a database query to make it 10x faster. The API inputs/outputs do not change.
2. You completely delete the `/api/legacy_stats` endpoint because nobody uses it anymore.
3. You add an optional `?format=json` parameter to the `/api/move` endpoint.

Answers

1) **v1.4.3** (Patch: Bug fix/Optimization)
2) **v2.0.0** (Major: Breaking change! Clients using that endpoint will crash)
3) **v1.5.0** (Minor: New feature, but old clients are unaffected)

Git Tagging

Once you decide on a SemVer number, a common release workflow is to attach it to a specific Git commit. This is called a **Git Tag**.

```
# 1. Ensure you are on the main branch
git checkout main

# 2. Create an annotated tag with a message
git tag -a v1.5.0 -m "Added optional JSON formatting"

# 3. Push the tag to GitHub
git push origin v1.5.0
```

Why? A Git commit hash (`a8f9c2e`) is meaningless to a human. A tag (`v1.5.0`) creates a permanent, readable bookmark in your history.

Agenda

Continuous Delivery vs. Continuous Deployment

We often hear “CI/CD”, but the “CD” can mean two different things.

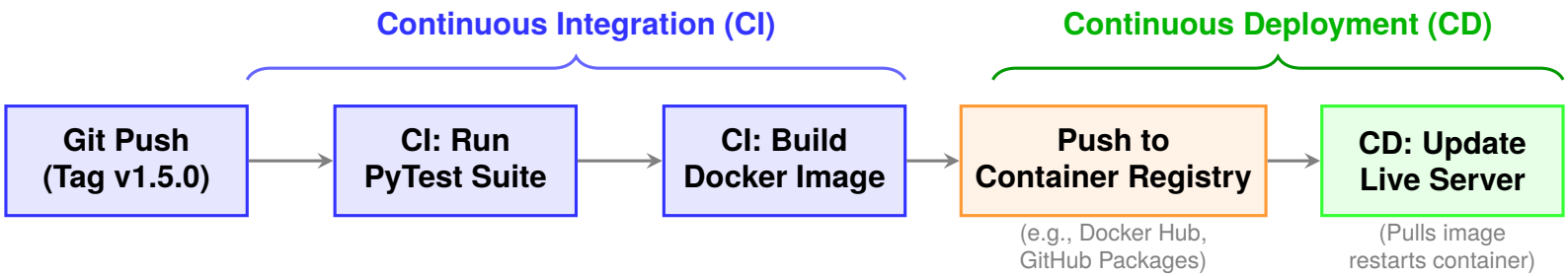
Continuous Delivery

The pipeline builds, tests, and packages the code into a releasable artifact (e.g., a Docker Image). It places it in a registry. **Deployment to live servers often includes a manual approval step, depending on team policy and risk tolerance.**

Continuous Deployment

Every change that passes all stages of your production pipeline is released to your customers **automatically, with no human intervention**.

The CD Pipeline Architecture



- **Immutable Artifacts:** A common practice is to publish immutable artifacts (for example Docker images, binaries, or packages) rather than deploying ad-hoc source bundles.
- **Rollback:** Redeploying a previously versioned artifact (for example image tag `v1.4.2`) is usually straightforward.
- **Note:** In our module example, this pipeline is triggered by a Git Tag; teams may also trigger from merges, release branches, or approval gates.

Agenda

The “Big Bang” Deployment (The Old Way)

How it works:

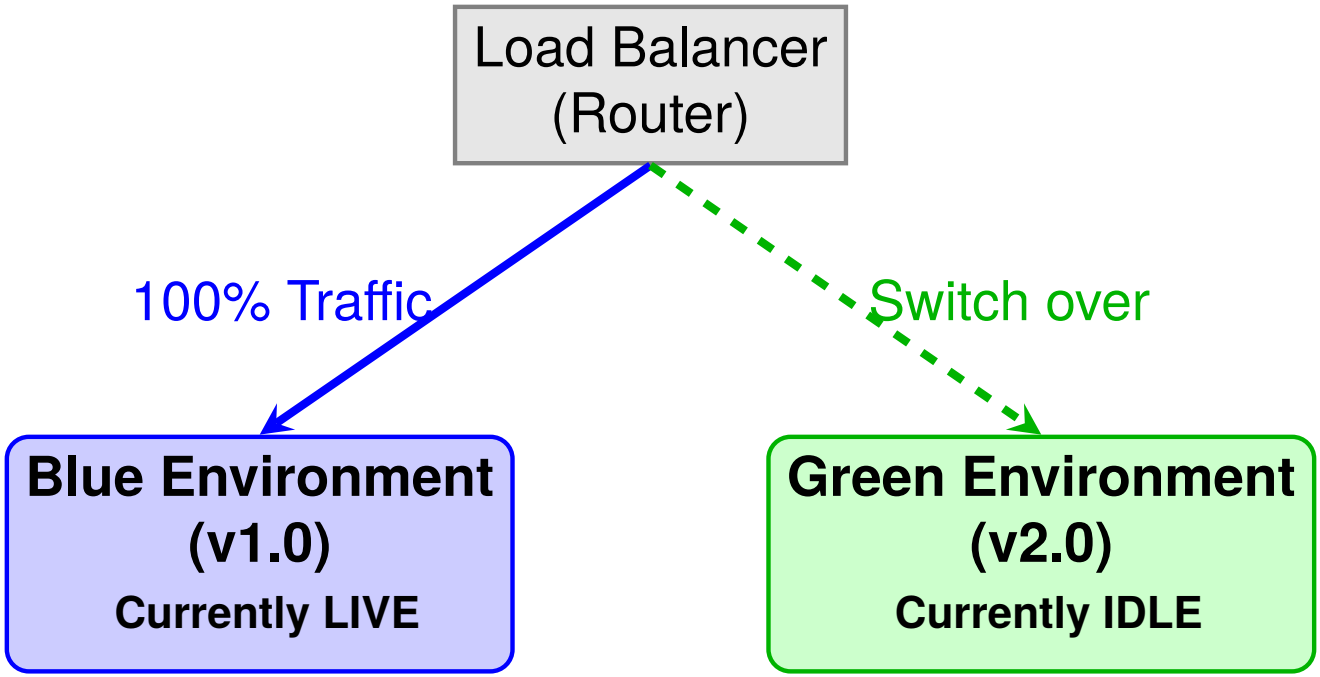
1. Put up a “Site Under Maintenance” page.
2. Turn off Server V1.
3. Install Server V2.
4. Turn Server V2 on and hope it works.

The Problem

Requires downtime. If V2 crashes immediately, you have to turn everything off again to rollback. This is unacceptable for modern applications (e.g., Netflix, Banking, Robot Control).

Blue/Green Deployment

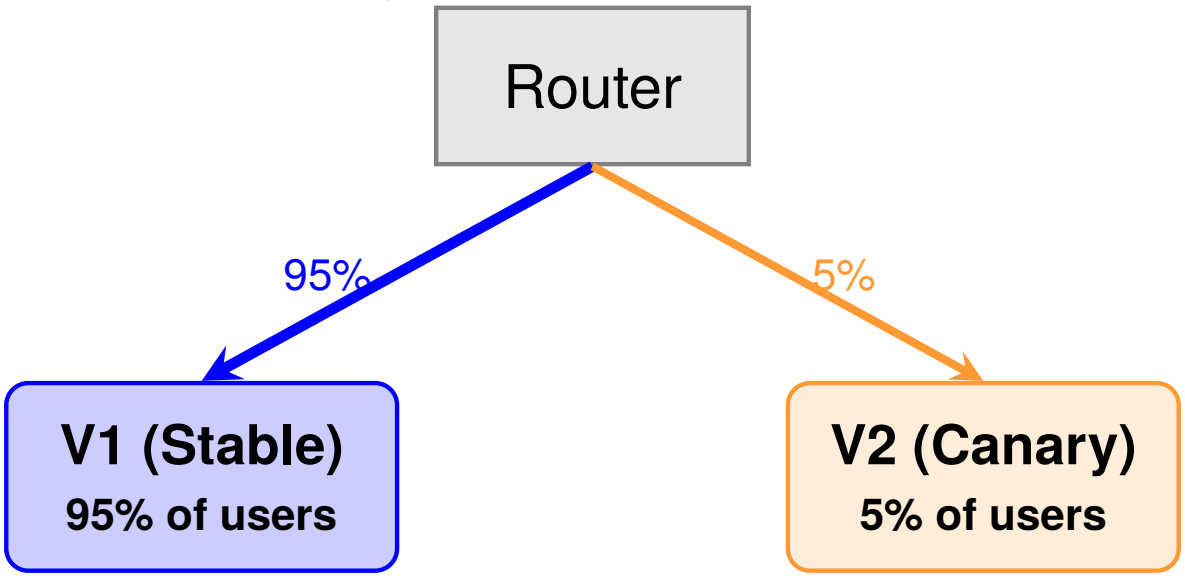
Goal: Zero Downtime Deployments.



- Spin up an exact clone of your production environment (Green).
- Deploy V2 to Green. Run tests on it *while it is invisible to users*.
- When ready, flip the Load Balancer router. Blue becomes idle, Green becomes live. Zero downtime. Instant rollback if things break.

Canary Releases

Named after the “Canary in a Coal Mine”.



- Monitor V2 error logs for 1 hour. No crashes? Dial up to 20%, 50%, 100%.
- **Benefit:** A catastrophic bug in V2 only affects **5%** of users, not 100%.

Canary vs. Blue/Green: When to Use Each?

Situation	Blue/Green	Canary
Database schema changes	**Often preferred**	**Higher risk**
Gradual confidence building	Harder	**Well-suited**
Instant rollback needed	**Fast (single switch)**	Slower (dial back)
Infrastructure cost	Higher (2 envs)	Lower
User-facing A/B testing	Less common	**Common**

Discussion (2 min)

Your GCS needs to deploy a breaking database schema change tonight. Which strategy do you choose, and why? What extra steps would you need?

Knowledge Check: Choose the Strategy

Scenario: You are deploying an update to the Ground Control Station that fundamentally alters the database schema (changing how coordinates are stored).

Question: In a shared-database setup, which deployment strategy is typically the *highest risk* to use here?

1. A) Blue/Green Deployment
2. B) Canary Release
3. C) Big Bang Deployment

Knowledge Check: Choose the Strategy

Scenario: You are deploying an update to the Ground Control Station that fundamentally alters the database schema (changing how coordinates are stored).

Question: In a shared-database setup, which deployment strategy is typically the *highest risk* to use here?

1. A) Blue/Green Deployment
2. B) Canary Release
3. C) Big Bang Deployment

Answer: B (Canary Release)

Canary releases run V1 and V2 **simultaneously**. If V2 writes new schema data to the database, V1 will crash because it doesn't understand the new format! Database-breaking changes usually require Blue/Green (with careful DB replication) or a brief Big Bang maintenance window.

Agenda

Deployment ≠ Release

One of the most powerful concepts in modern DevOps is decoupling the act of *deploying* code from the act of *releasing* a feature.

- **Deploying:** Putting the code on the production server.
- **Releasing:** Making that code visible/active for the end user.

How do we push a half-finished feature to production servers without breaking the application for users? **Feature Flags (Toggles).**

Implementing Feature Flags

Feature flags are often implemented as conditionals around new code paths, controlled by external configuration (for example env vars, config files, or a flag service).

```python
import os

# Read the flag from environment variables (defaults to False)
ENABLE_NEW_DASHBOARD = os.getenv("FF_NEW_DASHBOARD", "false").lower() == "true"

@app.get("/api/dashboard")
def get_dashboard_data():
    if ENABLE_NEW_DASHBOARD:
        # The new V2 code you are currently working on
        return fetch_advanced_telemetry()
    else:
        # The old, stable V1 code that users currently see
        return fetch_basic_telemetry()
```

The Power: Deploy this to production many times a day. When ready to launch, change the environment variable to `"true"`. **No redeployment needed!**

Feature Flag Pitfalls: Flag Debt

Feature flags are powerful, but they come with a hidden cost you should recognise from last week.

Common pitfalls:

- **Flags that never get cleaned up** — the old code path stays in the repo indefinitely, adding complexity (Lehman's 2nd Law!).
- **Testing explosion** — with n flags, you theoretically have 2^n code paths to test.
- **Flag coupling** — flags that depend on other flags create hidden interactions and bugs.

Rule of Thumb

When a flag is fully rolled out, **schedule its removal** in the same sprint. Treat it like any other item in your Technical Debt Register.

How They Combine in Practice

Deployment mechanics and feature flags are strongest when used together.

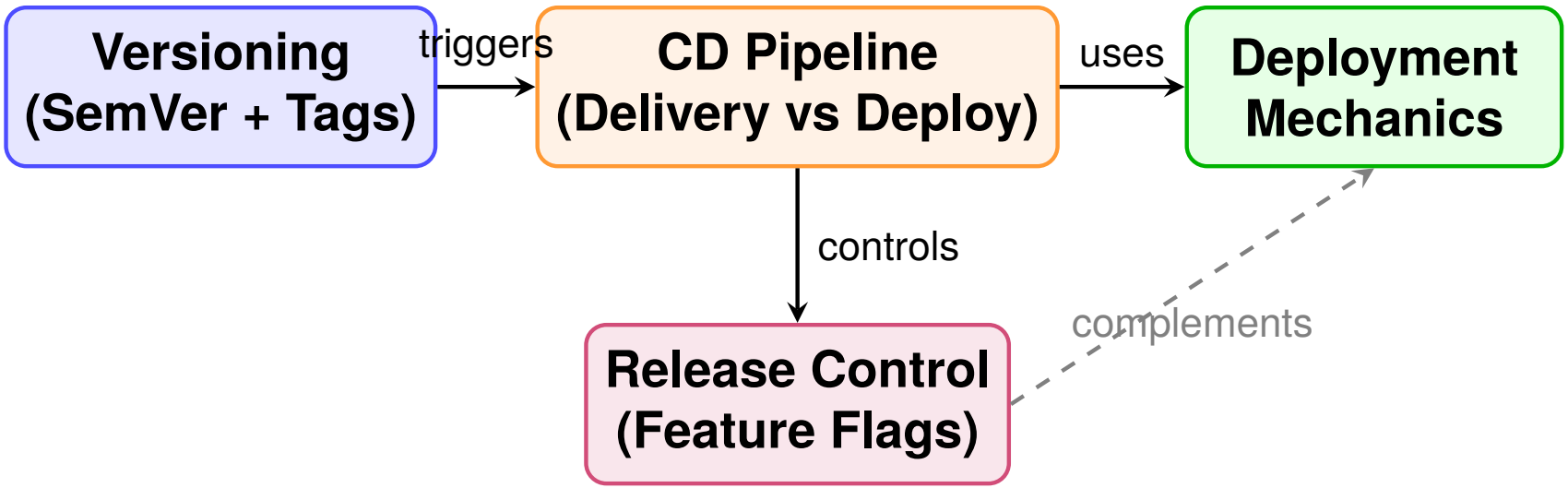
- **Canary + Flags:** Send 5% traffic to V2, but keep risky features off by default. Enable per cohort as confidence grows.
- **Blue/Green + Flags:** Switch infrastructure instantly, then progressively enable new features without redeploying.
- **Dark Launch + Flags:** Deploy code to production, keep UI hidden, and validate behavior/metrics before public release.

Operational Safety Net

Keep a **kill switch** flag for high-risk functionality so you can disable exposure quickly while investigating incidents.

Agenda

Lecture Summary: The Full Picture



Together these practices make deployments **traceable, safe, and invisible to end-users** — the goal from slide 5.

This Week's Lab: Release Management

In the workshop, you will practice managing releases for your Ground Control Station.

- **Task 1: Semantic Versioning:** Determine the correct SemVer numbers for various scenarios in your project.
- **Task 2: Git Tagging & GitHub Releases:** Create formal annotated tags and generate a changelog using GitHub Releases.
- **Task 3: Feature Flags:** Implement an environment-variable-driven Feature Flag in your backend API to hide a newly developed feature.

Next Lecture

LEPSI, Security & Professional Practice

Q & A

Any Questions?

fdelduchetto@lincoln.ac.uk